Stateless Service Design

any instance, any request, nothing lost when it dies

Watch the 4-min video

In one line: a stateless service keeps no client or request state in its own memory or disk between requests; sessions, files, caches, data and background work live in external stores (Redis, S3, databases, queues), so any instance can serve any request and instances are disposable, which makes scaling, deploys and failures simple and safe.

1 Gone with the pod

A user uploads a profile photo, and it's saved to the pod's local disk. Ten minutes later Kubernetes reschedules that pod, and the photo is gone. Another user's session lived in memory on a pod that just crashed, and they're logged out. The fix for both is the same: make the service **stateless**.

2 What stateless means

A stateless service keeps **no client or request state** in its own memory or disk **between requests**. Every request carries, or can look up, everything needed to handle it (a token, an ID). So any instance can serve any request, and instances can be added, removed or replaced at any time without losing anything.

Statelessness doesn't mean there's no state. It means the state lives **somewhere else**: sessions in Redis or a signed token, uploaded files in object storage like S3, caches in a shared cache, durable data in a database, background work in a queue. The service becomes a disposable process that reads and writes external state.

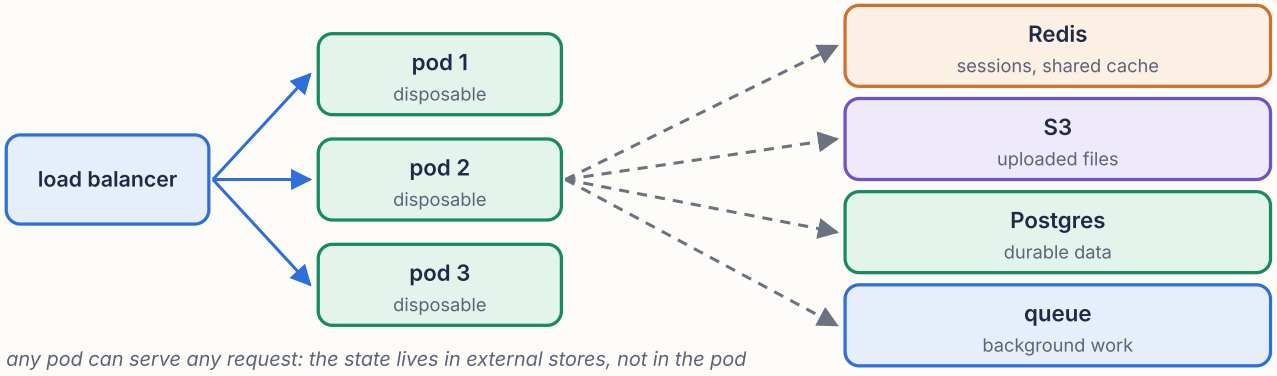


Figure 1. A stateless tier: identical, disposable pods in front of external stores.

3 The twelve-factor app

The **twelve-factor app** methodology captures this: processes are stateless and share nothing; configuration comes from the environment; backing services (databases, caches, queues) are attached resources; and processes are disposable: they start fast and shut down gracefully, finishing in-flight requests when they receive SIGTERM (lesson 7).



Figure 2. Four twelve-factor ideas that make a service stateless and disposable.

4 Idempotent handlers

Stateless handlers should be **idempotent** where possible. A client might retry after a timeout, and the retry might land on a different instance. With an **idempotency key** stored in the database (Phase 2, API lesson 39), the second attempt returns the original result instead of charging a card twice. Retries become safe no matter which instance handles them.

WORKED EXAMPLE: A FASTAPI APP ON MANY PODS

Originally, uploads were written to `/tmp` on the pod. Now the API generates a **presigned S3 URL**, the client uploads directly to S3, and the pod only stores the object key in Postgres. Sessions move to Redis, and local caches become short-lived optimisations only. The pods can now scale from 3 to 30, and be killed at any moment, with nothing lost. Bonus: large uploads no longer pass through the API at all.

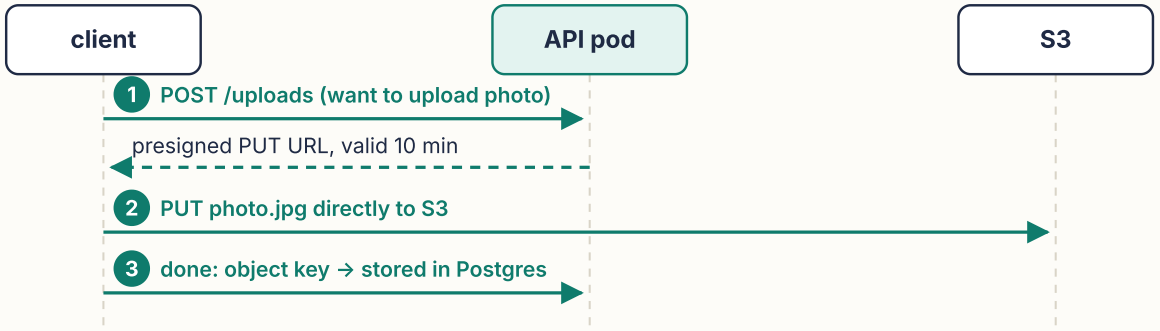


Figure 3. Presigned uploads: the API hands out a short-lived URL, the client sends the file straight to S3, the API stores only the key.

5 The payoff

- **Horizontal scaling is trivial**: a new instance is identical to the others (Phase 1, lesson 7).
- **Autoscaling, rolling deploys and spot instances** are safe (lessons 13 and 14).
- **Any load-balancing algorithm** works, with no sticky sessions (lesson 6).
- **Failures are cheap**: a dead instance loses nothing that matters.

COMMON MISTAKES

- Writing uploads, generated reports or SQLite files to the container's disk.
- In-memory sessions or rate-limit counters, so behaviour changes with which pod you hit.
- Scheduled jobs (cron) inside every web pod, so they run N times.
- Ignoring SIGTERM, so every deploy drops in-flight requests.

6 Key terms

Stateless service: no client state kept between requests

Disposable process: starts fast, stops gracefully

Presigned URL: short-lived URL to upload or download directly

External state: Redis, S3, databases, queues

Twelve-factor app: methodology for cloud-friendly services

Idempotency key: makes a retried write return the first result

7 Interview questions

Q1. What makes a service stateless? BASIC

It keeps no client state locally between requests: sessions, files, caches and data live in external stores like Redis, S3 and databases, so any instance can handle any request and instances are disposable.

Q2. Why does it matter? BASIC

It makes horizontal scaling, autoscaling, rolling deploys and failure recovery simple and safe.

Q3. Users lose uploaded files after deploys. What do you change? SCENARIO

Stop writing to local disk: upload to object storage (ideally directly with presigned URLs), store only the object key in the database, and serve files from S3 or a CDN.

Q4. How do you make a POST /payments handler safe to retry on any instance? SCENARIO

Require an idempotency key, store it with the result in the database inside the same transaction as the payment, and return the stored result on repeats.

Q5. Is a service with a local in-memory cache still stateless? TRICKY

Yes, if the cache is only an optimisation: losing it costs a few misses, never correctness, and every instance can rebuild it from the shared stores.

Q6. What does 'disposable' require in practice? TRICKY

Fast startup, readiness checks, handling SIGTERM by failing readiness and draining in-flight work, and no state that would be lost if the process is killed at any moment.

Next: SD-J-002 · Scaling Stateful Services backendroadmap.com